

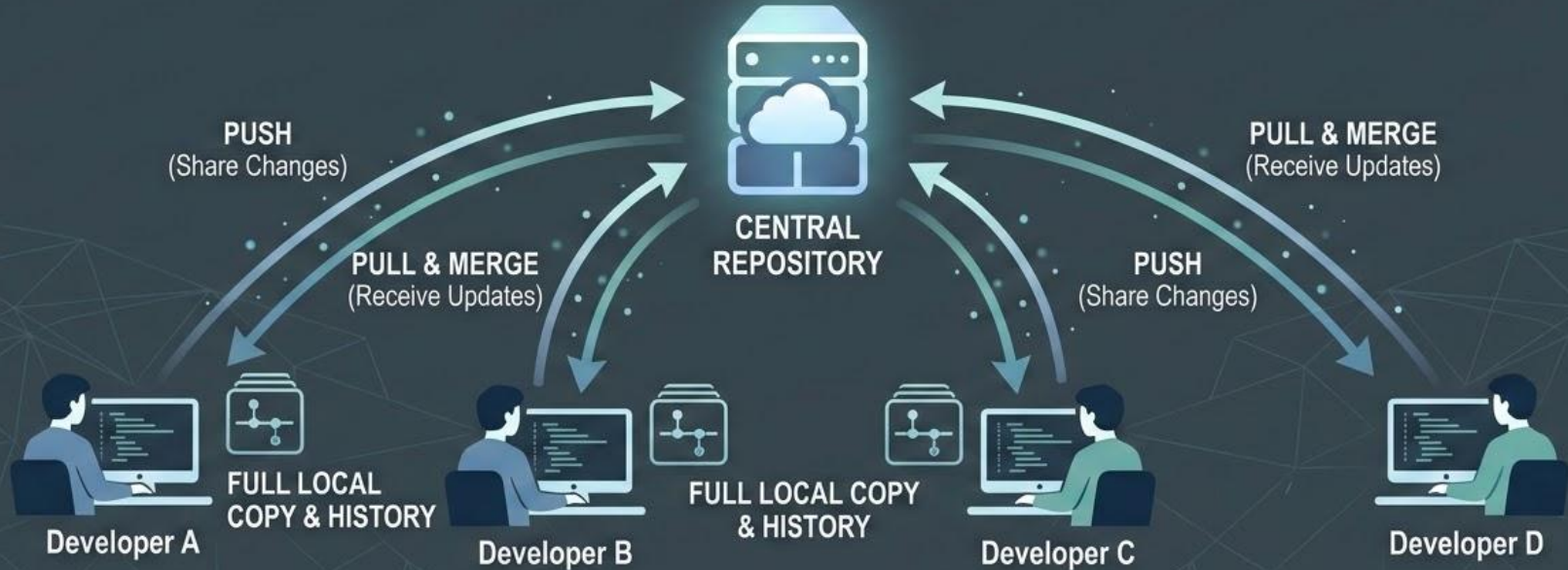
Version Control System

A version control system (VCS), also known as a source control system, is a software tool that helps software developers manage changes to source code over time. It allows developers to track changes to the code, collaborate on development, and maintain a history of all changes made to the codebase.



What is DVCS

A Distributed Version Control System (DVCS) gives every developer a *full* local copy of the cobase and history.



DVCS: KEY ADVANTAGES & GIT EXAMPLE



FLEXIBILITY & OFFLINE WORK

Work anywhere with less reliance on a central server.

FULL LOCAL REPO COPY



REDUNDANCY & DATA SECURITY

Full copy on each developer's machine reduces risk of data loss.



PARALLEL COLLABORATION

Eases team development, parallel work on branches, and seamlessly merging their changes together.

EXAMPLE - GIT



git

the de facto industry standard

WHAT IS GIT?



TRACK CHANGES

Git helps track changes made to a codebase over time.



KEY HISTORY

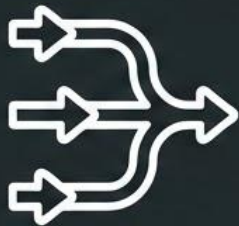
Created by Linus Torvalds in 2005 for managing the Linux kernel.



DISTRIBUTED COLLABORATION

Allows multiple developers to work simultaneously, each with their own copy of the repository. Changes are committed locally and merged seamlessly.

KEY FEATURES & INDUSTRY ADOPTION



POWERFUL BRANCHING & MERGING

Work on separate **branches** to test new features. **Merge** changes back into the main project safely and easily.



INDUSTRY DE FACTO STANDARD

The **de facto standard** for version control. Free and **open-source**. Runs on **Windows**, **macOS**, and **Linux**.

Commands

-  **git init:** initializes a new Git **repository** in a directory
-  **git add:** adds files to the **staging area** (the area where changes are prepared to be committed)
-  **git commit:** saves changes to the **local repository** with a commit message
-  **git clone:** creates a **copy** of a **remote repository** on your local machine
-  **git pull:** updates your **local repository** with changes from a remote repository
-  **git push:** pushes changes from your **local repository** to a remote repository
-  **git branch:** lists or creates **branches** in the repository

Advanced Commands



git checkout: switches between different **branches** or **commits** in the **repository**



git merge: merges changes from one **branch** into another



git status: shows the **status** of the **working directory** and the **staging area**



git log: shows a **history** of **commits** in the repository

What is Github



GIT HOSTING

Web-based platform providing hosting for Git repositories. Manage and view codebase efficiently.



TEAM COLLABORATION

Enables individuals and teams to work on projects simultaneously. View changes and create new branches easily.



INTEGRATED MANAGEMENT

Features for tracking issues, managing bugs, and using pull requests to merge changes into the main codebase.

One of the most important features of GitHub is its social aspect. Developers can follow other developers and projects, and contribute to open-source projects hosted on the platform. This has helped to create a vibrant community of developers who share their knowledge and work together to create better software.

Branches

Branches in Git allow you to work on different versions of your project, test new things, and then combine them back into the main project when you're ready.

In Git, you merge branches to combine the changes made in one branch with another branch, typically bringing the changes from a feature branch into the main branch (often called the "master" or "main" branch). Merging is important for a few reasons:

Why merge

1. Incorporate New Features: When you work on a new feature or fix a bug in a feature branch, you want to bring those changes into the main branch so that your project benefits from the improvements. Merging allows you to add these new features or fixes to the main branch.
2. Keep the Project Up-to-Date: Projects evolve, and multiple people may be working on different branches simultaneously. Merging helps you keep the main branch up-to-date with the latest changes, ensuring that everyone's work is integrated smoothly.

3. Maintain a Clean History: Merging helps you maintain a clean and organized project history. Each merge represents a point where a set of changes was incorporated into the main branch, making it easier to track the progress of the project.

4. Collaboration: Git is often used for collaborative software development. Merging allows team members to share their work and collaborate on different aspects of a project while keeping everything in sync.

Pull Request

A pull request is a request to “pull” your changes into another branch.

It’s used when you collaborate with others on the same project — before merging, your teammates can review, discuss, and approve the changes.

<https://docs.github.com/en/get-started/using-github/github-flow>

Steps

Create a new branch – e.g. feature/login-page

Make changes – write code, fix bugs, etc.

Commit and push your branch to GitHub.

Open a pull request – choose:

Base branch: where you want to merge (e.g. main)

Compare branch: your branch (e.g. feature/login-page)

Review process: teammates check code, suggest edits, and approve.

Merge the pull request – after approval, the changes go into the main branch.

Why It's Important

Encourages code review before merging

Helps track discussions about specific code changes

Ensures the main branch stays stable

Automatically triggers CI/CD pipelines (like GitHub Actions) if configured

Merge

Types of Git Merge

Fast-forward Merge

When the target branch has not diverged; Git just moves the pointer forward.

Three-way Merge

When both branches have new commits; Git creates a new merge commit

Switch to the branch you want to merge INTO

git checkout main

Merge the feature branch

git merge feature-branch

Always pull latest changes before merging:

Is git pull the same as git merge?

`git pull`

It performs

`git fetch + git merge`

Example

`git pull origin main`

If both have changes

If both branches (your local and remote) have changes, Git will do a three-way merge — and you may get merge conflicts to resolve.

`git fetch` Gets the latest changes from remote, but doesn't merge

`git merge` Merges a branch into your current branch

`git pull` Combines fetch + merge (automatically merges remote changes)

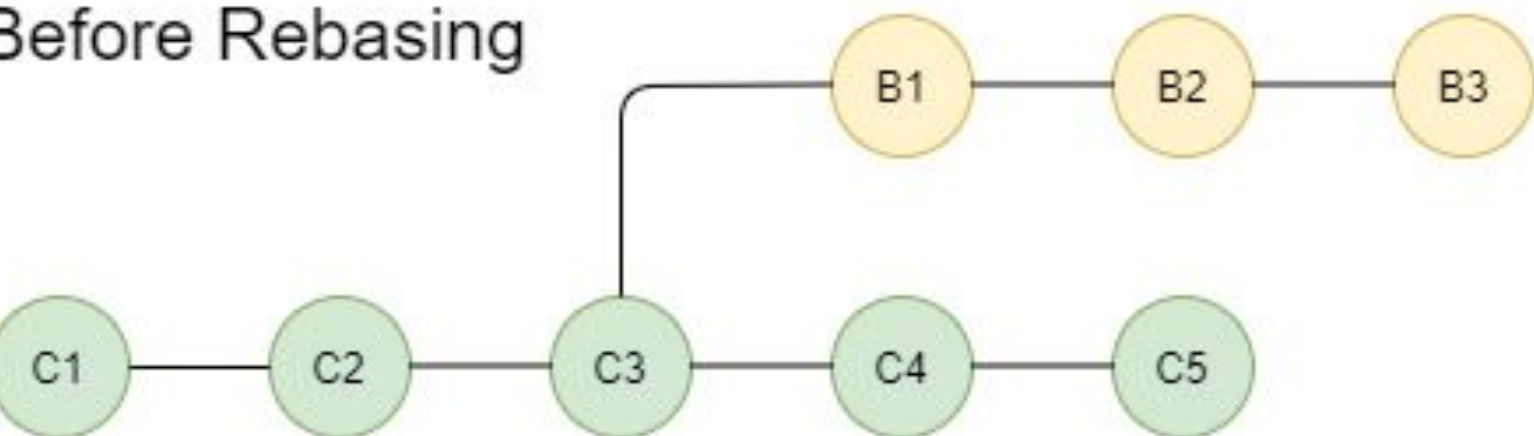
git rebase

git rebase moves your branch's commits on top of another base commit.

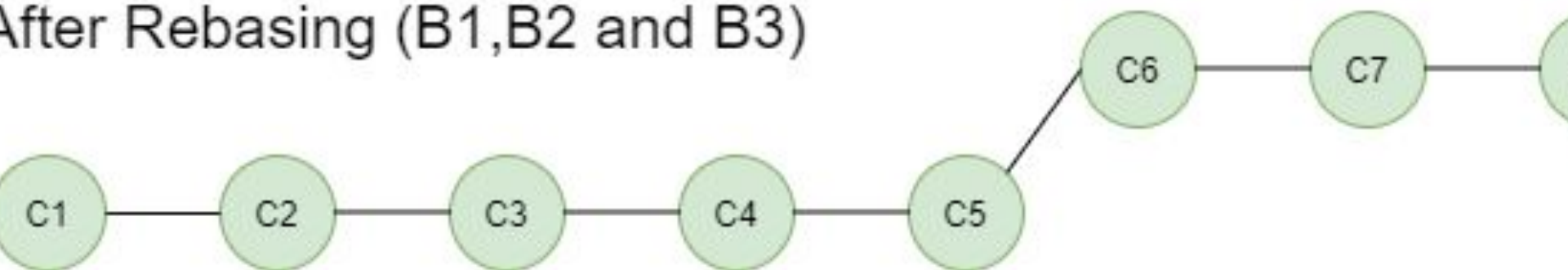
In simpler terms:

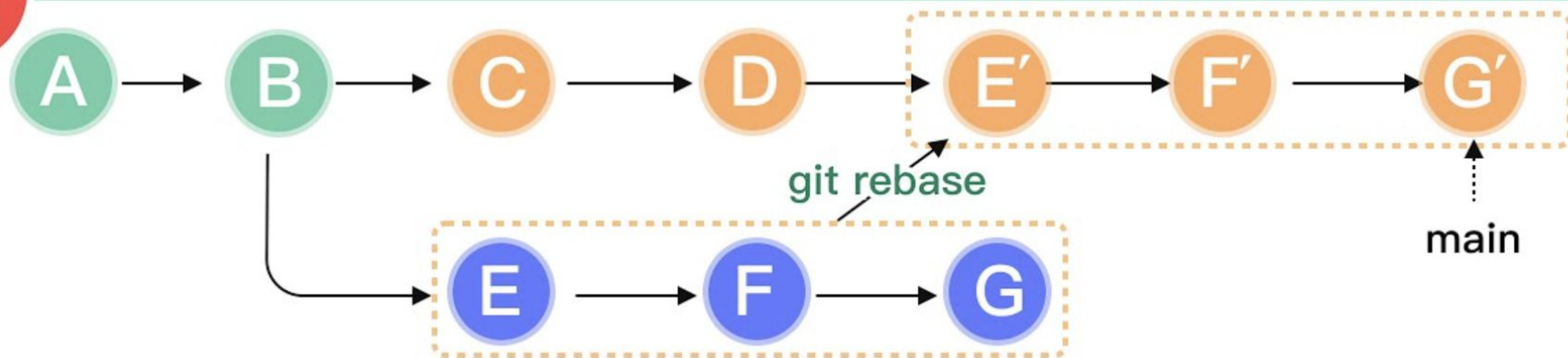
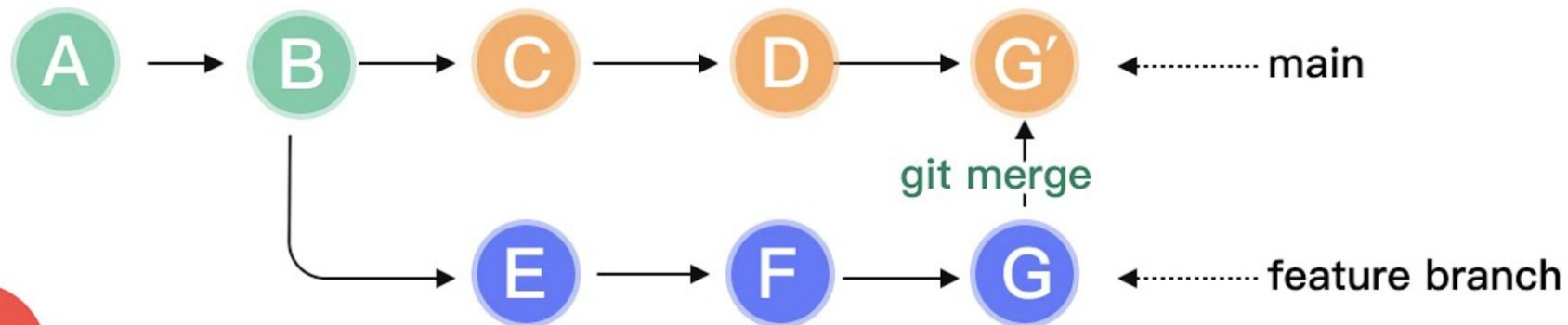
It replays your commits starting from a new point — as if you made them after the latest changes in another branch (like main)

Before Rebasing



After Rebasing (B1,B2 and B3)





Git revert

git revert undoes a commit by creating a new commit that reverses the changes from a previous one — without altering the commit history.

```
git revert <commit-hash>
```

Git reset

git reset moves the HEAD and optionally modifies your staging area or working directory.

It's mainly used to:

Undo commits

Unstage files

Discard local changes

```
git reset [--soft|--mixed|--hard] <commit>
```

